

一、填空题（每空格 1 分，共 20 分）

1. 求解 有限条 确定性
2. Θ 越小 越大
3. 优先队列 H[2i] H[2i+1]
4. 分治 $\Theta(n \log n)$ $\Theta(n^2)$ $\Theta(n \log n)$
5. 分治 解决冗余 存储中间计算结果
6. 较高 不能
7. 深度优先 广度/宽度优先

二、计算题(每题 10 分，共 20 分)

(1) 解：

$$T(n) = (n-1) + \frac{1}{n} \sum_{w=1}^n (T(w-1) + T(n-w))$$

$$\text{因为: } \sum_{w=1}^n T(n-w) = T(n-1) + T(n-2) + \cdots + T(0) = \sum_{w=1}^n T(w-1)$$

$$\text{所以: } T(n) = (n-1) + \frac{2}{n} \sum_{w=1}^n T(w-1)$$

$$n \cdot T(n) = n(n-1) + 2 \sum_{w=1}^n T(w-1) \dots\dots (a)$$

$$(n-1)T(n-1) = (n-1)(n-2) + 2 \sum_{w=1}^{n-1} T(w-1) \dots\dots (b)$$

(a) - (b)，并适当变换得：

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2(n-1)}{n(n+1)}$$

令 $D(n) = \frac{T(n)}{n+1}$ ，我们有：

$$D(n) = D(n-1) + \frac{2(n-1)}{n(n+1)}, D(1) = 0。 \text{进一步，可以得到：}$$

$$\begin{aligned} D(n) &= 2 \sum_{j=1}^n \frac{j-1}{j(j+1)} = 2 \sum_{j=1}^n \frac{2}{(j+1)} - 2 \sum_{j=1}^n \frac{1}{j} \\ &= 4 \sum_{j=2}^{n+1} \frac{1}{j} - 2 \sum_{j=1}^n \frac{1}{j} = 2 \sum_{j=1}^n \frac{1}{j} - \frac{4n}{n+1} = \Theta(\log n) \end{aligned}$$

所以有：

$$T(n) = (n+1)D(n) = \Theta(n \log n)$$

(2) 解：由题意，算法 A 的时间复杂度可以用如下的递归形式表达：

$$T(n) = 5 \cdot T(n-1) + 6 \cdot T(n-2) \quad (5 \text{ 分})$$

其特征方程为：

$$x^2 = 5 \cdot x + 6$$

解之，得两个根：

$$x_1 = 6, \quad x_2 = -1$$

因此， $T(n)$ 可以表示为：

$$T(n) = c_1 \cdot 6^n + c_2 \cdot (-1)^n$$

所以， $T(n) = \Theta(6^n)$ (5 分)

类似地，我们可以求得算法 B 的时间复杂度：

$$T(n) = \Theta(4^n)$$

所以，从时间复杂度的角度来讲，算法 B 较优。

三、解答题(共 60 分)

(1) false 2i i ← i+1 true 2i > n

(2) 解：

记 $f(n)$ 为 $n!$ 的值，那么：

$$f(n) = \begin{cases} n \cdot f(n-1), & n > 0 \\ 1, & n = 0 \end{cases}$$

因此，设计动态规划算法如下：

输入：整数 n ($n \geq 0$)

输出： $n!$ 的值

```
1. result ← 1
2. for i ← 0 to n
3.   result ← result * i
4. end
5. return result
```

时间复杂度为 $T(n) = \Theta(n)$

(3) 解：

(a) $F(i, j)$ 的递归定义如下：

$$F(i, j) = \begin{cases} 1, & \text{if } j == 1 \\ 1, & \text{if } i == j \\ j * F(i-1, j) + F(i-1, j-1), & \text{else} \end{cases}$$

(b) 输入： 集合包含的元素个数 n

输出： 集合的划分数

```

1.      for i ← 1 to n
2.          F(i,1) ← 1
3.          F(i,i) ← 1
4.      end
5.      for i ← 3 to n
6.          for j ← 2 to n-1
7.              F(i,j) ← j * F(i-1,j) + F(i-1,j-1)
8.          end
9.      end
10.     num ← 0
11.     for i ← 1 to n
12.         num ← num + F(n,i)
13.     end
14.     return num

```

时间复杂度为 $T(n) = \Theta(n^2)$

(4) 解：

(a)

$$D(i, j) = \min(D(i-1, j), D(i, j-1)) + v(i, j)$$

$$D(1,1) = v(1,1)$$

(b)

输入： 给定的 $n \times n$ 下三角矩阵 v

输出： 最小的路径值

```

1.  D(1,1) ← v(1,1)
2.  for i ← 2 to n
3.      D(i,1) = D(i-1,1) + v(i,1)

```

```

4.  end
5.  for i ← 2 to n
6.      for j ← 2 to i
7.          D(i,j) ← min( D(i-1,j), D(i,j-1)) + v(i,j)
8.      end
9.  end
10. result ← D(n,1)
11. for i ← 2 to n
12.     if result > D(n,i)
13.         result ← D(n,i)
14.     end
15. end
16. return result;

```

时间复杂度为 $T(n) = \Theta(n^2)$